AcadMate Shared Neon Database Setup Guide

This guide explains how team members can connect the current AcadMate FastAPI backend to the shared PostgreSQL development database hosted on Neon.

Current shared setup

Component	Current setup
Backend branch	feature/backend-integration
Backend framework	FastAPI
ORM	SQLAlchemy
Shared database	PostgreSQL on Neon
Verified endpoint	GET /api/v1/students/101/academic-profile
Expected result	200 OK with shared student academic data

Steps for each team member

1. Pull the latest integration branch

Open the local F2026-Team3 repository and pull the latest **feature/backend-integration** branch.

2. Go to the backend folder

```
cd backend
```

3. Create or activate a Python virtual environment

Use the existing local virtual environment if one already exists.

On Windows PowerShell, the existing environment can be activated with:

```
.\venv\Scripts\Activate.ps1
```

4. Install backend dependencies

```
pip install -r requirements.txt
```

5. Create a local backend/.env file

Add the shared Neon PostgreSQL connection string in this format:

```
DATABASE_URL=postgresql://...
```

The actual Neon DATABASE_URL should be shared privately by the database/backend owner.

6. Test the shared database connection

```
python test_db.py
```

Expected output: `Database connection successful: 1`

7. Start the FastAPI backend

```
uvicorn main:app --reload
```

8. Open Swagger API documentation

```
http://127.0.0.1:8000/docs
```

9. Test the student academic profile endpoint

Use:

```
GET /api/v1/students/101/academic-profile
```

A successful shared-database test should return **200 OK** and the student profile/enrollment data from Neon.

Important team rules

- Do not commit the **.env** file or Neon credentials to GitHub.
- Do not post the full Neon DATABASE_URL in public channels, Jira, or repository files. Share it privately.
- The shared schema and demo data are already created in Neon.
- Do not run **create_tables.py** or **seed_data.py** against the shared Neon database unless the team intentionally decides to update or reset the shared database.
- Use **feature/backend-integration** as the current backend integration branch.

Shared architecture: Local frontend/backend code -> FastAPI -> SQLAlchemy -> shared Neon PostgreSQL database.

For current team work

Li can test the FastAPI Router/Service/Pydantic integration against the shared database. Tafa can prepare the finalized synthetic CSV dataset using the agreed schema. Pranali can connect the student dashboard to the FastAPI endpoints instead of maintaining a separate permanent Node/Express backend.